

## Laboratoire de Programmation Temps Réel

Semestre printemps 2025 - 2026

### Laboratoire 4: Exploration de Xenomai 4/ EVL

Temps à disposition: 4 périodes

Laboratoire en groupe de 2 personnes

#### Objectifs pédagogiques

Ce laboratoire a pour objectif d'explorer les principaux services proposés par EVL, en mettant l'accent sur les mécanismes de synchronisation entre le domaine temps réel (RT) et le domaine non temps réel (non-RT), ainsi qu'entre tâches temps réel.

Les observations ainsi que les résultats obtenus au cours de vos différentes expériences doivent être consignés dans un rapport, comme pour les labos précédents.

#### Cross-buffer

Un problème très courant dans les systèmes à double noyau est d'échanger des données entre des threads *out-of-band* (temps réel) et *in-band* (non temps réel), sans introduire de délai du côté *out-of-band*.

Le cœur EVL implémente les *cross-buffers* (ou *xbuf*), qui relient l'extrémité émettrice *out-of-band* d'un canal de communication à son extrémité réceptrice *in-band*, et inversement.

Un exemple d'utilisation serait de permettre à une interface graphique (GUI) de recevoir des données de supervision provenant d'une boucle de travail temps réel.

`xbuf.c` fournit va permettre d'expérimenter l'utilisation des *cross-buffers*. Complétez ce programme pour implémenter les fonctionnalités suivantes :

1. Un *Cross-Buffer* permettant d'échanger des données depuis une tâche EVL vers une tâche Linux.
2. Une tâche EVL qui récupère périodiquement le temps courant et le stocke dans le *cross-buffer* créé au point précédent. La tâche EVL doit tourner sur le CPU 1 (CPU isolé)
3. Une tâche Linux qui lit les valeurs présentes dans le *cross-buffer* et affiche sur la sortie standard:
  - le temps courant,
  - la différence de temps entre deux lectures successives,
  - la valeur lue dans le *cross-buffer*,
  - la différence entre deux valeurs successives lues dans le *cross-buffer*.

Testez votre application, en mode normale et sous charge, en utilisant l'application `stress`. Qu'observez-vous ? Que pouvez-vous en déduire ?

## Proxy

---

Comme mentionné précédemment, un problème courant dans les systèmes à double noyau provient de la contrainte de ne pas effectuer d'appels système *in-band* lors de l'exécution de code critique en temps réel dans un contexte *out-of-band*.

Le problème est que, dans certains cas, il peut être nécessaire, d'écrire dans des fichiers ordinaires afin d'enregistrer sur la console ou ailleurs à partir d'une tâche *out-of-band*. Le faire en appelant directement les routines classiques `read` ou `write` n'est donc pas envisageable pour des raisons de latence.

Le cœur EVL résout ce problème grâce à la fonctionnalité de file proxy, qui permet d'envoyer des données vers un fichier cible arbitraire et/ou, inversement, de lire des données provenant de ce fichier cible, tout en fournissant une interface d'E/S *out-of-band* aux threads producteurs et consommateurs de données.

Un exemple d'utilisation consiste à enregistrer les valeurs provenant de capteurs. Vous allez réaliser un programme permettant d'expérimenter l'utilisation des *proxies*, qui simule la lecture de capteurs dans un contexte *out-of-band*. Les valeurs mesurées seront enregistrées dans un fichier de journalisation (*log*).

Pour cela, complétez le fichier `proxy.c` fourni. Ce programme devra implémenter les fonctionnalités suivantes :

1. Une tâche EVL qui simule la lecture de deux capteurs. Ces capteurs retournent des valeurs correspondant à deux signaux sinusoïdaux décalés dans le temps et ayant des amplitudes différentes. La tâche lit périodiquement les valeurs de ces capteurs.
2. Une tâche Linux chargée de la gestion du fichier de journalisation : ouverture du fichier et écriture des données obtenues des capteurs. Lors de chaque écriture, les deux valeurs doivent être écrites sur la même ligne, séparées par un espace.
3. L'utilisation d'un *proxy* pour l'échange des données entre les contextes *in-band* et *out-of-band*.

Testez votre application, en mode normale et sous charge, en utilisant l'application `stress`. Qu'observez-vous ? Que pouvez-vous en déduire ?

Le script `plot_results.py` fourni peut être utilisé pour afficher, sous forme graphique, les valeurs des capteurs enregistrées dans le fichier `log`. Le script attend que le fichier se nomme `sensors.log` et qu'il soit présent dans le même répertoire.

Pour récupérer le fichier généré, vous pouvez utiliser la commande `scp`. Veuillez vous référer au laboratoire 1 pour plus de détails sur l'utilisation de cette commande.

La figure 1 montre un exemple du résultat de l'utilisation du script `plot_results.py`.

## Surveillance d'un système multi-tâches par Watchdog

---

Dans un système embarqué critique (médical, automobile ou aéronautique), le respect des échéances temporelles est aussi vital que l'exactitude des calculs. Une tâche dépassant son temps d'exécution alloué (gigue excessive, boucle infinie, interblocage) peut compromettre le bon fonctionnement de l'ensemble du système.

Un groupe de *flags d'événements* constitue un mécanisme efficace et léger permettant de synchroniser plusieurs threads. Il repose sur une valeur de 32 bits, où chaque bit représente l'état d'un événement spécifique défini par l'application.

L'objectif de ce laboratoire est de compléter le fichier `flags.c` afin de mettre en œuvre un mécanisme de supervision de type *Watchdog*, capable de surveiller plusieurs tâches temps réel s'exécutant en

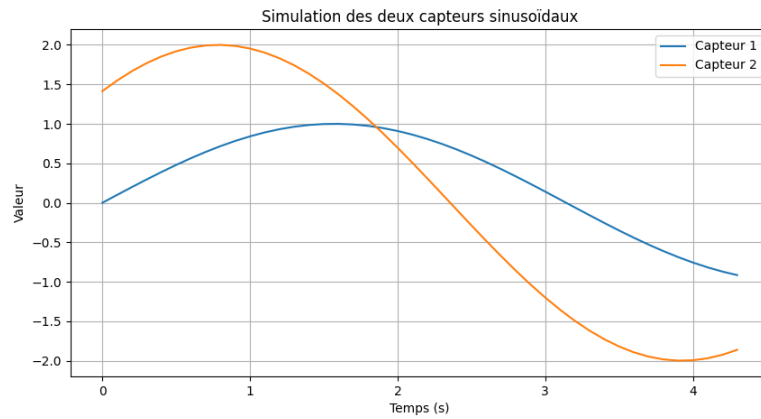


Figure 1: Exemple des capteurs depuis le fichier sensors.log

parallèle sur un noyau Linux/EVL, en utilisant les `evl flags`.

Les `evl flags` seront également utilisés pour implémenter une barrière de synchronisation garantissant que tous les threads sont prêts avant le début du traitement.

Le système se compose de deux tâches *Worker*, simulant une charge de travail périodique avec une gigue (jitter) contrôlée. À chaque cycle, ces tâches doivent signaler leur bon fonctionnement au Watchdog. Si l'une d'elles échoue à signaler sa complétion avant l'échéance, le Watchdog doit lever une alerte, identifier la tâche fautive et réinitialiser la surveillance pour le cycle suivant.

## Objectifs

- Implémenter les Workers et le Watchdog sous forme de threads *out-of-band* (temps réel)
- Utiliser les `evl flags` pour synchroniser le démarrage des Workers et du Watchdog
- Mettre en œuvre un mécanisme de surveillance basé sur les `evl flags`, permettant de vérifier que chaque tâche termine son cycle dans une fenêtre temporelle prédéfinie
- En cas de dépassement de délai (*timeout*), identifier le thread fautive via l'inspection de l'état du groupe de flags